

Software technology practice

A guide to evaluate the work of the students

This is an example of the evaluation criterias. The evaluation can be done individually (0-5, 0-10 or other scale). More experienced practice teachers can also evaluate the teams (e.g. on a scale of 0-15 or 0-30 in the case of a 3-person team) and offer that the team members can decide for themselves how to distribute the score they receive.

1st Milestone

Requirement analysis and design: feasibility plan, functional and non-functional specification, use-case diagrams, user stories, class diagram, component diagram, user interface design, database schema diagram (if there is a DB).

Aspects, advices:

- The functional specification and the **use-case diagram** should reflect each other's content, both describe the functions of the application. There may be differences, of course, the use-case diagram is user (actor)-centered and helps to visually show the relationships between the individual functions. It is suspicious if there are use cases on the use-case diagram that do not appear in the functional specification.
- The use-case diagram is not a workflow diagram. It is not intended to describe the internal processes of the user or the program, the sequence diagram and the state diagram should be used for that. (These were already listed on OOP, but will also appear on Software Technology.)
- **User stories** are closely related to testing. The user stories are sufficiently detailed if based on them the (manual) testing of the application could be performed by a non-programming user. You also need to pay attention to edge cases, and to handle malfunctions properly, you need to write a user story. In the development phase, well-written user stories also guarantee that all team members understand the same operation of each function.
- The **user interface plan** should be prepared with some target program (mock-up designer). Show as many screens and states as possible (e.g. popup sample).
- Regarding the **class diagram**, the public interface of the classes and the more important (private) representation are basically expected, all private auxiliary procedures do not need to be indicated. A good class diagram shows the responsibilities and relationships of each type. Failure to do so is a design shortcoming that, during implementation, may result in the team members' work not matching each other properly, then new design decisions have to be made, which entail the refactoring of the already written codebase, causing extra work.
- Creating dynamic design diagrams is not mandatory, but for example, the sequence diagram, which describes the communication between components, is suitable for modeling longer, more complicated processes. This can also help team members to gain a more thorough understanding of the operation of the program to be implemented during the planning phase.

Presentation:

1. By the time of the presentation, all documents should already be on the Wiki page (it is advisable to edit them here in advance).
2. Since there is not much time for the presentation, the goal is not a reading of the documents, but a targeted, essential presentation of the completed work (regardless of this, the completed materials will be reviewed in detail for the evaluation in addition to the presentation). It is worth preparing a script, in which order and what you will present.

Evaluation criteria:

- Quality of finished works
- Division of work, team communication (to what extent the entire team understands and knows the completed documents, later code / program)
- Continuity of week-by-week progress
- Using the expected functions of the project management service (GitLab).

2nd Milestone

Prototype 1: partial implementation, approx. 30% of the use cases should work; using Git

Evaluation aspects:

1. Issue management (*issue board* on Git)
 - Tasks should be added and tracked regularly and continuously
 - During the presentation, the completed tasks should be in a closed state
 - The *board* should be consistent with the work performed (the task should be assigned to the right person(s))
2. Git usage
 - Use commits and branches regularly
 - Commit messages should be meaningful
 - Proper use of .gitignore (unnecessary files and folders, e.g. bin and obj are not committed)
 - The presented code should be in the master branch
3. Functionality: Approx. 30% of use cases are implemented
 - This should cover the entire application in horizontal, so the model and view layers should also have completed functions, the game can be tested
 - It is possible to create a prepared "demo mode" for the presentation, so that individual functions can be presented more easily.
4. Code
 - Code quality is basically only evaluated at the 4th milestone, but it is advisable to organize and write the code according to the principles of *Clean Code* from the beginning, there will be less work with it later.
 - The implementation should not contain architectural errors ("we will fix it later").
5. Documentation
 - If the implementation deviated from the design, it does not mean a negative in the evaluation, however, the specification must follow the implementation, so in this case, as previously discussed, it is also expected to be updated for the presentation (wiki).
 - The requests indicated during the first milestone should be improved.

Presentation:

1. Prepare for the presentation with a script and presentation sequence. Each team will have 10 minutes in time.
2. The current state of the program should be presented, with all its working functions (this can be done by one person, but you can also split it up.)
3. The presentation should reveal which functions and tasks each team member was responsible for.
4. We do not ask for a detailed description of the code at the presentation, but we do ask for a description of at least one part that is thought to be more exciting. (E.g. what was realized differently compared to the plans and why, or what was a more difficult problem to solve and how was it solved?)
5. Briefly address what changes you needed in the documentation (if any).

3rd Milestone

Prototype 2: partial implementation, approx. 90% of the use cases should work; using Git, CI; partial unit-testing

Evaluation aspects:

1. Issue management is still important
2. Functionality:
 - The program must be in an almost complete state, which means that the basic task has been implemented in its entirety, and the sub-functions must also be almost complete. The functionality of at most 1 subtask should be in a non-trialable state.
 - The visualization (rendering) should be appropriate (the content should not hang out of the window, the fields should be displayed correctly, etc.). It is best if there are already graphics (pictures).
 - The program should be user-friendly and provide clear feedback on incorrect cases.
3. Tests: the model should be tested as fully as possible, the ultimate goals in this are:
 - Correct operation with error handling must also be tested (happy path testing and error path testing)
 - Let there be tests for edge-cases
 - There should be tests for difficult-to-reproduce situations
 - It is worth looking at code coverage, which shows how much of the code is covered by tests. In general, a value above 70-80% is considered medium (below low), a value above 90% as good/high, and a value above 95% as outstanding. By the end of the semester, we aim for an 80% code coverage for the model.
 - Milestone 3 should have at least 3 non-trivial test cases (constructor and getter/setter testing is considered trivial).
4. CI: The pipeline builds the project and runs the tests
 - The pipeline should run at least for every push of the master branch.
5. Documentation: Any changes in plans should be reported in the Wiki (e.g. class diagram)

Presentation: Same as for the second milestone

- Prepare the event in such a way that, if possible, you can easily present all the functions. If necessary, several pre-prepared situations can be pre-defined (e.g. in a file, or in a separate branch, etc.) for easier presentation.

4th Milestone

Final product: full functionality, clean-code, documented, tested (thorough unit testing)

Evaluation aspects:

1. Issue management is still important
2. Functionality:
 - All required and assumed functions of the program should be fully functional.
 - The application should be fault-tolerant, properly handle any incorrect user input or interaction.
 - The application should be easy to use: following software ergonomic principles with conventional placement of controls, helping the user with position-sensitive or application-level help so that the user can try out all the functions of the game without knowing the source code. This aspect also includes fine-tuning the parameters of the game: the game should be neither too easy nor too difficult.
3. Code:
 - The architecture should be followed properly.
 - The principles of *Clean code* should be applied (there should not be too long functions and classes, the SOLID principles should be implemented).
 - The warnings indicated by the compiler (with special attention to null warnings) should be resolved (it is not a solution if the null-state analysis is turned off, this should be enabled everywhere).
 - The source code should use a uniform coding convention.
4. Tests:
 - All functionality of the model should be fully tested, with a specific emphasis on *error paths* and *edge cases*. As a goal, we define 80% testing of the model as a minimum in terms of code coverage.
 - Tests should preferably be divided into different test files by topic.
5. CI/CD: See the previous milestone
6. Documentation:
 - All documentation should be fully completed, with a clear emphasis on the specification and class diagram (the latter one can be exported from the IDE).
 - API documentation of the classes and methods of the program is expected (at least for the public and protected interfaces). In case of Java, use JavaDoc. In case of C#, use XML-doc. These tools let you generate a HTML manual.

Presentation:

- All functions of the program should be fully presented. It is still worth preparing with pre-specified cases to make the presentation easier.
- It is no longer necessary to present the code, but if there is something extra interesting or a solution that you would definitely like to show to others, then there is an opportunity to do so.
- In the case of documentation, you could provide information on how well the original plans were kept and how many changes were needed.